

۱. زیرساخت و راه‌اندازی اولیه

پایه و اساس کار ما، راه‌اندازی سرویس‌های مورد نیاز به صورت مجزا و مدیریت شده در یک محیط داکری است. برای این منظور، سرویس‌های `elasticsearch` و `kibana` را به فایل `docker-compose.yml` اضافه شده است.

• سرویس `Elasticsearch`:

- از ایمج رسمی `elasticsearch:8.11.3` استفاده شده است.
- برای سادگی در محیط توسعه، حالت `discovery.type` را برابر `single-node` قرار داده شده و نیازی به تنظیمات کلاسترینگ پیچیده نبود.
- گزینه `xpack.security.enabled=false` برای غیرفعال کردن ویژگی‌های امنیتی `X-Pack` (مانند نیاز به نام کاربری و رمز عبور) تنظیم شده تا ارتباط بین سرویس جنگو و `Elasticsearch` در شبکه داخلی داکر به راحتی برقرار شود.
- پورت 9200 کانینر به پورت 9200 هاست متصل شده تا بتوانیم مستقیماً با API آن در ارتباط باشیم.

• سرویس `Kibana`:

- این سرویس به عنوان یک ابزار مدیریتی و مانیتورینگ برای `Elasticsearch` عمل می‌کند و روی پورت 5601 در دسترس است.
- از طریق متغیر محیطی `ELASTICSEARCH_HOSTS` به سرویس `elasticsearch` متصل می‌شود.

برای اینکه اپلیکیشن جنگو ما بتواند با `Elasticsearch` ارتباط برقرار کند، متغیر محیطی `ELASTICSEARCH_HOST=elasticsearch` را برای سرویس `web` در `docker-compose.yml` تعریف کردم. این کار باعث می‌شود که در کدهای پایتون، با استفاده از نام سرویس (`elasticsearch`) به راحتی به آن دسترسی داشته باشیم.

۲. فرآیند ایندکس‌گذاری داده‌ها

مهم‌ترین قدم برای قابل جستجو کردن داده‌ها، انتقال آن‌ها از پایگاه داده اصلی (`PostgreSQL`) به `Elasticsearch` است. این فرآیند که ایندکس‌گذاری نام دارد، از طریق یک دستور مدیریتی سفارشی در جنگو به نام `sync_elastic` انجام می‌شود.

این اسکریپت که در مسیر `api/management/commands/sync_elastic.py` قرار دارد، وظایف زیر را به ترتیب انجام می‌دهد:

1. اتصال به Elasticsearch: با استفاده از کتابخانه `elasticsearch-py` و خواندن متغیر محیطی `ELASTICSEARCH_HOST`، یک کلاینت برای ارتباط با سرور ایجاد می‌کند.
2. مدیریت ایندکس:
 - ابتدا بررسی می‌کند که آیا ایندکسی به نام `tickets` وجود دارد یا خیر.
 - اگر وجود داشت، آن را حذف می‌کند. این کار تضمین می‌کند که هر بار با اجرای دستور، داده‌های قدیمی و ناسازگار پاک شده و داده‌های جدید جایگزین شوند.
 - سپس یک ایندکس خالی با همان نام (`tickets`) ایجاد می‌کند.
3. تجمیع داده‌ها از PostgreSQL: یک کوئری SQL جامع و بهینه نوشته شده که با استفاده از `LEFT JOIN` بین جداول `tickets`, `locations`, `vehicles` و جداول مربوط به هر نوع وسیله نقلیه (`flights`, `buses`, `trains`)، تمام اطلاعات مورد نیاز برای هر بلیط را در یک ردیف استخراج می‌کند.
4. آماده‌سازی داده‌ها برای ایندکس:
 - داده‌های خام دیتابیس به دیکشنری‌های پایتون تبدیل می‌شوند.
 - فیلدهای `NULL` حذف می‌شوند تا حجم سندها در Elasticsearch کاهش یابد.
 - فیلدهای مربوط به امکانات (`facility`) که در دیتابیس به صورت رشته JSON ذخیره شده‌اند، به آبجکت‌های JSON واقعی تبدیل می‌شوند تا در آینده بتوان بر روی آن‌ها نیز جستجوهای دقیق‌تری انجام داد.
5. ایندکس‌گذاری دسته‌ای (Bulk Indexing): به جای ارسال تک‌تک بلیط‌ها به Elasticsearch، تمام آن‌ها را در یک لیست از "عملیات" (`actions`) جمع‌آوری کرده و با استفاده از تابع `helpers.bulk` از کتابخانه `elasticsearch-py`، همه را در یک درخواست به سرور ارسال می‌شود. این روش سرعت فرآیند ایندکس‌گذاری را افزایش می‌دهد و بار کمتری روی شبکه و سرور ایجاد می‌کند.

۳. جزئیات پیاده‌سازی API جستجو

پس از آماده شدن داده‌ها در ایندکس `tickets`، ویوی `search_tickets_view` در فایل `api/views.py` وظیفه جستجو را بر عهده می‌گیرد.

- ساخت کوئری داینامیک: بر اساس ورودی کاربر، یک کوئری `bool` در `Elasticsearch` ساخته می‌شود که از دو بخش اصلی تشکیل شده است:

- `must`: این بخش برای شرایطی است که حتماً باید برقرار باشند. برای مثال، `origin_city` و `destination_city` در این بخش قرار می‌گیرند. استفاده از `match` به جای `term` در اینجا به ما انعطاف‌پذیری بیشتری می‌دهد و می‌تواند جستجوهای تمام متن (Full-text search) را نیز پشتیبانی کند. برای `company_name` نیز از `multi_match` استفاده کرده‌ام تا همزمان در فیلدهای `airline_name` و `company_name` جستجو انجام شود.
- `filter`: این بخش برای فیلتر کردن نتایج بر اساس شرایط دقیق و بدون محاسبه امتیاز (score) است که باعث افزایش سرعت جستجو می‌شود. فیلترهایی مانند `ticket_status`، `vehicle_type`، `departure_date` و محدوده قیمت (`price`) در این بخش قرار می‌گیرند. برای فیلدهایی که نیاز به تطابق دقیق دارند (مانند `flight_class.keyword`) از `term` استفاده شده است.

این رویکرد ترکیبی (استفاده همزمان از `must` و `filter`) باعث می‌شود که هم بتوانیم جستجوهای مرتبط و امتیازبندی شده داشته باشیم و هم از سرعت بالای فیلتر کردن بهره‌مند شویم.

۴. همگام‌سازی و به‌روزرسانی لحظه‌ای داده‌ها

یک سیستم جستجو تنها زمانی کارآمد است که داده‌های آن به‌روز باشد. از آنجایی که اجرای مجدد دستور `sync_elastic` برای هر تغییر کوچک (مانند کم شدن ظرفیت یک بلیط) منطقی نیست، یک راهکار برای به‌روزرسانی لحظه‌ای پیاده‌سازی کرده‌ام.

- تابع `update_ticket_in_elastic`: این تابع که در `api/elastic_utils.py` قرار دارد، `ticket_id` و یک دیکشنری از تغییرات را به عنوان ورودی می‌گیرد و با استفاده از متد `update` کلاینت `Elasticsearch`، سند مربوط به آن بلیط را به‌روز می‌کند.
- فراخوانی در نقاط حساس: این تابع در نقاطی از کد که داده‌های مهم بلیط تغییر می‌کنند، فراخوانی می‌شود. به عنوان مثال، در ویوی `reserve_ticket_view`، پس از اینکه یک صندلی با موفقیت رزرو موقت می‌شود و ظرفیت باقی‌مانده (`remaining_capacity`) یک واحد کاهش می‌یابد، این تابع فراخوانی شده و مقدار جدید را در سند `Elasticsearch` نیز به‌روز می‌کند.

Python

```
transaction.on_commit(  
    lambda: update_ticket_in_elastic(ticket_id, {"remaining_capacity":  
new_remaining_capacity})  
)
```

استفاده از `transaction.on_commit` تضمین می‌کند که این به‌روزرسانی تنها در صورتی انجام می‌شود که تراکنش دیتابیس با موفقیت به پایان رسیده باشد و از عدم هماهنگی بین دو پایگاه داده جلوگیری می‌کند.

این رویکرد، یکپارچگی داده‌ها را در سطح بالایی حفظ کرده و تضمین می‌کند که نتایج جستجو همیشه منعکس‌کننده وضعیت واقعی سیستم هستند.

```
class Command(BaseCommand):  
    help = 'Syncs tickets data from PostgreSQL to Elasticsearch for searching'  
  
    def handle(self, *args, **options):  
        self.stdout.write("Connecting to Elasticsearch...")  
        es_host = os.environ.get("ELASTICSEARCH_HOST", "localhost")  
        try:  
            es_client = Elasticsearch(  
                hosts=[{"host": es_host, "port": 9200, "scheme": "http"}]  
            )  
            if not es_client.ping():  
                raise ConnectionError("Could not connect to Elasticsearch cluster.")  
        except Exception as e:  
            self.stderr.write(self.style.ERROR(f"Elasticsearch connection failed:  
{e}"))  
  
            return  
  
        index_name = "tickets"  
        self.stdout.write(f"Preparing index '{index_name}'...")  
  
        if es_client.indices.exists(index=index_name):  
            self.stdout.write(f"Deleting existing index '{index_name}'...")  
            es_client.indices.delete(index=index_name)  
  
        es_client.indices.create(index=index_name)  
        self.stdout.write(f"Index '{index_name}' created successfully.")  
  
        query = """  
        SELECT  
            t.ticket_id, t.price, t.total_capacity, t.remaining_capacity,  
t.ticket_status,  
            t.departure_start, t.departure_end, t.is_round_trip,  
            origin.city AS origin_city,  
            origin.province AS origin_province,  
            dest.city AS destination_city,  
            dest.province AS destination_province,  
            v.vehicle_type,  
            f.airline_name, f.flight_class, f.number_of_stop, f.flight_code,  
            f.origin_airport, f.destination_airport, f.facility AS flight_facility,  
            tr.train_stars, tr.choosing_a_closed_coupe, tr.facility AS train_facility,  
            b.company_name, b.bus_type, b.number_of_chairs, b.facility AS bus_facility  
        FROM tickets t
```

```

LEFT JOIN locations origin ON t.origin_location_id = origin.location_id
LEFT JOIN locations dest ON t.destination_location_id = dest.location_id
LEFT JOIN vehicles v ON t.vehicle_id = v.vehicle_id
LEFT JOIN flights f ON v.vehicle_id = f.vehicle_id
LEFT JOIN trains tr ON v.vehicle_id = tr.vehicle_id
LEFT JOIN buses b ON v.vehicle_id = b.vehicle_id;
"""

self.stdout.write("Fetching data from PostgreSQL...")
with connection.cursor() as cursor:
    cursor.execute(query)
    columns = [col[0] for col in cursor.description]
    fetched_rows = cursor.fetchall()
    self.stdout.write(f"Fetched {cursor.rowcount} rows from the database.")

    actions = []
    for row in fetched_rows:
        row_dict = dict(zip(columns, row))

        source_doc = {k: v for k, v in row_dict.items() if v is not None}

        for facility_key in ['flight_facility', 'train_facility',
'bus_facility']:
            if facility_key in source_doc and
isinstance(source_doc[facility_key], str):
                try:
                    source_doc[facility_key] =
json.loads(source_doc[facility_key])
                except json.JSONDecodeError:
                    source_doc[facility_key] = {} # Default to empty object
on error

                if "ticket_id" not in source_doc:
                    self.stderr.write(self.style.WARNING(f"Skipping a row because it
has no ticket_id: {row_dict}"))
                    continue

                actions.append({
                    "_index": index_name,
                    "_id": source_doc["ticket_id"],
                    "_source": source_doc
                })

    if not actions:
        self.stdout.write("No documents to index.")
        return

    self.stdout.write(f"Attempting to index {len(actions)} documents into
Elasticsearch...")
    try:
        success_count, errors = helpers.bulk(es_client, actions,
raise_on_error=False)

        self.stdout.write(self.style.SUCCESS(f'Successfully indexed
{success_count} documents.'))

        if errors:
            self.stderr.write(self.style.ERROR(f'Encountered {len(errors)} errors
during indexing:'))
            for i, error in enumerate(errors):
                if i >= 5:
                    self.stderr.write(self.style.ERROR("... and more errors."))
                    break

```

```

        self.stderr.write(f" - Error: {error}")

    except Exception as e:
        self.stderr.write(self.style.ERROR(f'A critical error occurred during the
bulk indexing operation: {e}'))

```

```

def update_ticket_in_elastic(ticket_id: int, updates: dict):
    """
    Updates a specific ticket document in Elasticsearch.

    Args:
        ticket_id: The ID of the ticket to update.
        updates: A dictionary containing the fields to update.
                  Example: {"remaining_capacity": 19}
    """
    try:
        es_host = os.environ.get("ELASTICSEARCH_HOST", "localhost")
        es_client = Elasticsearch(
            hosts=[{"host": es_host, "port": 9200, "scheme": "http"}]
        )
        if es_client.ping():
            es_client.update(index="tickets", id=ticket_id, doc=updates)
            print(f"Successfully updated ticket {ticket_id} in Elasticsearch with:
{updates}")
        except NotFoundError:
            print(f"Warning: Ticket with ID {ticket_id} not found in Elasticsearch. Could
not update.")
        except Exception as e:
            print(f"ERROR: Failed to update ticket {ticket_id} in Elasticsearch: {e}")

```

```

@csrf_exempt
@require_http_methods(["POST"])
def search_tickets_view(request):
    """
    Searches for tickets using Elasticsearch based on various criteria.

    This endpoint receives a JSON payload with search filters and constructs a
dynamic query to search the 'tickets' index in Elasticsearch. It supports
required fields like origin, destination, and date, as well as a wide
range of optional filters for vehicle type, price, company, and specific
vehicle attributes.

    Request Body (JSON):
    {
        "origin_city": "Tehran",
        "destination_city": "Mashhad",
        "departure_date": "2025-07-10",
        "vehicle_type": "BUS",                // Optional: 'FLIGHT', 'TRAIN', 'BUS'
        "min_price": 100000,                  // Optional
        "max_price": 500000,                  // Optional
        "company_name": "Hamsafar",          // Optional (for Flight/Bus)
        "flight_class": "Economy",           // Optional (for Flight)
    }

```

```

        "train_stars": 4,                // Optional (for Train)
        "bus_type": "VIP"               // Optional (for Bus)
    }

    Successful Response (JSON - Status Code: 200 OK):
    {
        "status": "success",
        "data": [
            {
                "ticket_id": 22,
                "origin_city": "Tehran",
                "destination_city": "Qazvin",
                // ... other ticket details
            }
        ],
        "cached": false
    }

    Error Responses (JSON):
    - 400 Bad Request: For invalid JSON, missing required fields, or bad data formats.
    - 500 Internal Server Error: For database or unexpected server errors.
    - 503 Service Unavailable: If the connection to Elasticsearch fails.
    """
    try:
        data = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({'status': 'error', 'message': 'Invalid JSON format.'},
            status=400)

    if not all(k in data for k in ['origin_city', 'destination_city',
        'departure_date']):
        return JsonResponse({
            'status': 'error',
            'message': 'Missing required parameters: origin_city, destination_city,
departure_date'
        }, status=400)

    try:
        es_host = os.environ.get("ELASTICSEARCH_HOST", "localhost")
        es_client = Elasticsearch(
            hosts=[{"host": es_host, "port": 9200, "scheme": "http"}]
        )
        if not es_client.ping():
            raise ConnectionError("Could not connect to Elasticsearch")
    except Exception as e:
        print(f"Elasticsearch connection error: {e}")
        return JsonResponse({'status': 'error', 'message': 'Search service is
temporarily unavailable.'}, status=503)

    search_query = {
        "query": {
            "bool": {
                "must": [],
                "filter": [
                    {"term": {"ticket_status": True}}
                ]
            }
        }
    }

    search_query["query"]["bool"]["must"].append({"match": {"origin_city":
data["origin_city"]}})
    search_query["query"]["bool"]["must"].append({"match": {"destination_city":

```

```

data["destination_city"]]))

    try:
        date_str = data['departure_date']
        start_of_day = f"{date_str}T00:00:00"
        end_of_day = f"{date_str}T23:59:59"
        search_query["query"]["bool"]["filter"].append({
            "range": {
                "departure_start": {
                    "gte": start_of_day,
                    "lte": end_of_day,
                    "format": "yyyy-MM-dd'T'HH:mm:ss"
                }
            }
        })
    except (ValueError, KeyError):
        return JsonResponse({'status': 'error', 'message': 'Invalid date format for
departure_date (YYYY-MM-DD).'},
                            status=400)

    if data.get("vehicle_type"):
        search_query["query"]["bool"]["filter"].append({"match": {"vehicle_type":
data["vehicle_type"].upper()}})

    price_range_query = {}
    if data.get("min_price"):
        price_range_query["gte"] = data["min_price"]
    if data.get("max_price"):
        price_range_query["lte"] = data["max_price"]
    if price_range_query:
        search_query["query"]["bool"]["filter"].append({"range": {"price":
price_range_query}})

    if data.get("company_name"):
        search_query["query"]["bool"]["must"].append({
            "multi_match": {
                "query": data["company_name"],
                "fields": ["airline_name", "company_name"],
                "fuzziness": "AUTO"
            }
        })

    if data.get("flight_class"):
        search_query["query"]["bool"]["filter"].append({"term":
{"flight_class.keyword": data["flight_class"]}})

    if data.get("train_stars"):
        search_query["query"]["bool"]["filter"].append({"term": {"train_stars":
data["train_stars"]}})

    if data.get("bus_type"):
        search_query["query"]["bool"]["filter"].append({"term": {"bus_type.keyword":
data["bus_type"]}})

    try:
        response = es_client.search(index="tickets", body=search_query, size=100) #
Increase size to get more results
        results = [hit['_source'] for hit in response['hits']['hits']]
        return JsonResponse({"status": "success", "data": results, "cached": False})
    except Exception as e:
        print(f"Elasticsearch search error: {e}")
        return JsonResponse({'status': 'error', 'message': 'An error occurred during
the search.'}, status=500)

```